Assignment No : 1

Name: Atharv Gade **Roll No:** COB413

Title : Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS .

```
import concurrent.futures from

collections import deque


class Graph:      def __init__(self, vertices):

self.vertices = vertices          self.graph = {i: []

for i in range(vertices)}


    def add_edge(self, u, v):

self.graph[u].append(v)

self.graph[v].append(u)  # For undirected graph


# Parallel BFS def

parallel_bfs(graph, start):

visited = [False] * graph.vertices

visited[start] = True     queue =

deque([start])     result = []


    while queue:

level_size = len(queue)

level_nodes = []


        # Process all nodes at the current level in parallel
        with concurrent.futures.ThreadPoolExecutor() as executor:
```

```
futures = []            for _ in range(level_size):                node = queue.popleft()
level_nodes.append(node)                # Submit task for processing neighbors of the
node                futures.append(executor.submit(process_bfs, node, graph, visited,
queue))

        # Wait for all tasks to finish            for future in
concurrent.futures.as_completed(futures):                future.result()

      result.extend(level_nodes)

   return result

def process_bfs(node, graph, visited, queue):
for neighbor in graph.graph[node]:   if not
visited[neighbor]:            visited[neighbor] =
True           queue.append(neighbor)

# Parallel DFS def
parallel_dfs(graph, start):
visited = [False] * graph.vertices
result = []

   # Start DFS using ThreadPoolExecutor for recursive calls
with concurrent.futures.ThreadPoolExecutor() as executor:
      futures = [executor.submit(dfs_recursive, graph, start, visited, result)]
for future in concurrent.futures.as_completed(futures):
         future.result()

   return result
```

```
def dfs_recursive(graph, node, visited, result):
if visited[node]:
        return
visited[node] = True     result.append(node)

    # Parallelize the exploration of neighbors     with
concurrent.futures.ThreadPoolExecutor() as executor:         futures = []         for neighbor
in graph.graph[node]:             if not visited[neighbor]:
futures.append(executor.submit(dfs_recursive, graph, neighbor, visited, result))

        # Wait for all futures to complete         for future in
concurrent.futures.as_completed(futures):
            future.result()

# Example Usage if
__name__== "__main__":
    # Create a graph with 6 vertices (0 to 5)
graph  =  Graph(6)   graph.add_edge(0,  1)
graph.add_edge(0, 2)     graph.add_edge(1,
3)     graph.add_edge(1, 4)
graph.add_edge(2, 5)

    # Parallel BFS starting from node 0
start_node = 0
    print("Parallel BFS starting from node 0:")
bfs_result = parallel_bfs(graph, start_node)
print(bfs_result)
```

```
    # Parallel DFS starting from node 0
print("\nParallel DFS starting from node 0:")
dfs_result = parallel_dfs(graph, start_node)
print(dfs_result)
```

OUTPUT :

```
Parallel BFS starting from node 0:
[0, 1, 2, 3, 4, 5]
Parallel DFS starting from node 0:
[0, 1, 2, 3, 4, 5]
```